RICE UNIVERSITY

NCCL and GPU Communication

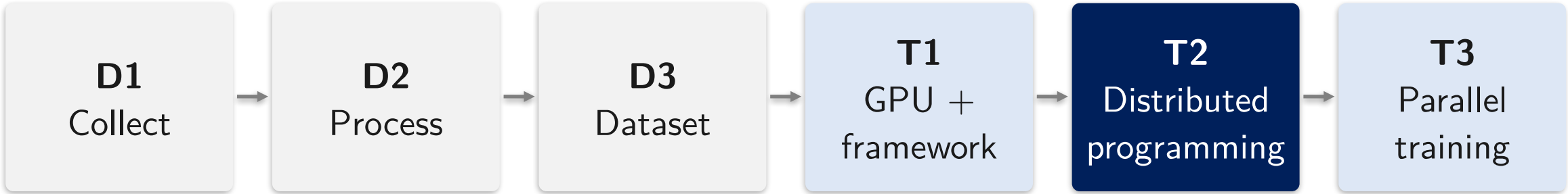
How GPUs add up their gradients, how fast that can be, and how to test it

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 11, Lecture 1

Where we are in the pipeline



In our pipeline

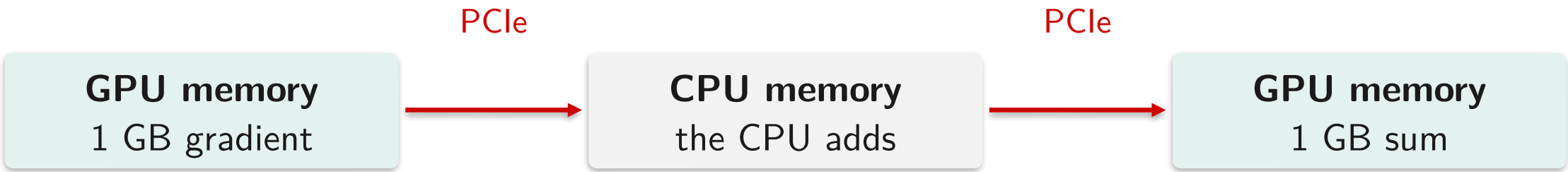
Today: collectives **between GPUs** with NCCL. T2 checkpoint this week: correct collectives on small tensors.

Last time: processes, ranks, collectives on CPUs, the α–β model. Next time: many servers.

Why do we need NCCL?

8 GPUs in one server. After backward, each holds 1 GB of gradients.
Before the next step, every GPU needs their **sum**.

Without a GPU library, the sum goes through the CPU. How long?



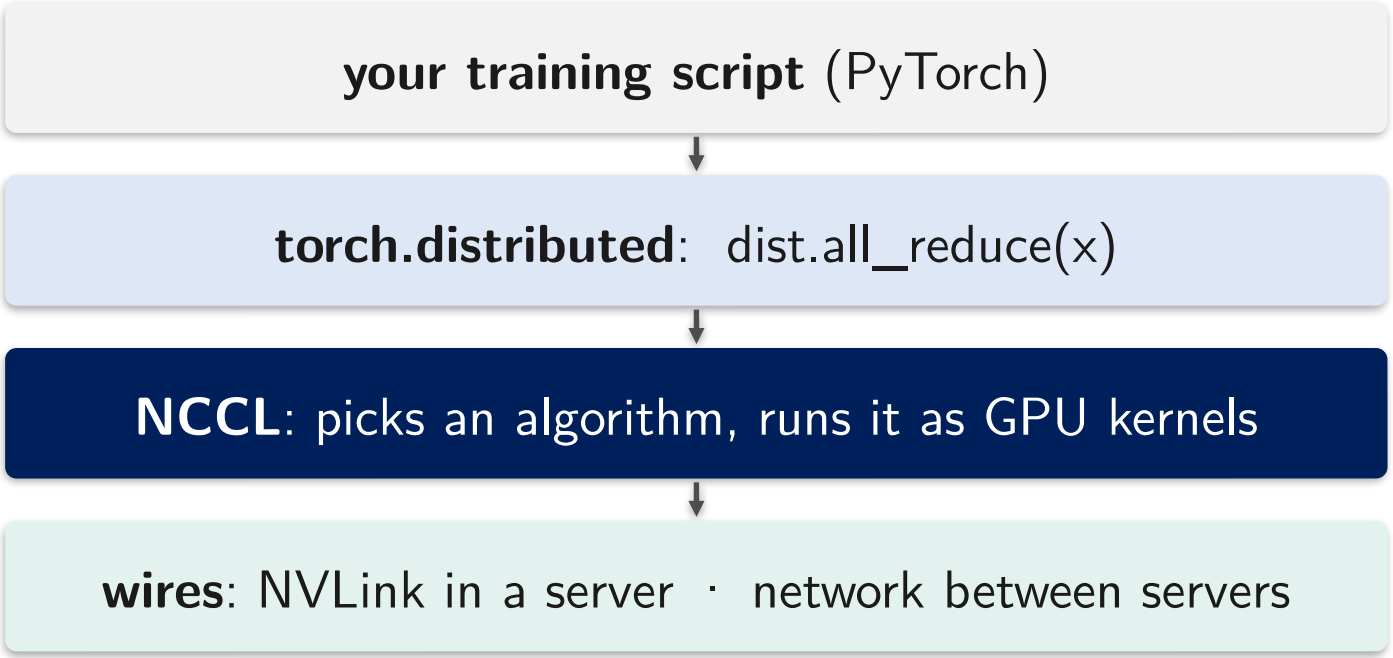
PCIe $\approx$ 64 GB/s each way, crossed twice by every GPU

Answer

$\geq$ **31 ms** per step just moving bytes — while the CPU adds 8 GB and the fast GPU-to-GPU wires sit **idle**.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). PCI-SIG PCIe 5.0.

What NCCL is, in plain words



- **library**: code you call, not write.
- **kernel**: a program that runs on the GPU.
- Data **stays in GPU memory**; the GPUs do the adding.

Definition

NCCL ("nickel"): collectives written as **GPU programs**, sending bytes over the fastest wire — NVLink $\approx$ 450 GB/s each way.

Source: H100 datasheet: NVLink 4, 900 GB/s bidirectional per GPU = 450 GB/s per direction. NVIDIA NCCL User Guide, Overview.

Where NCCL shows up

- **PyTorch**: the GPU backend of torch.distributed. DDP and FSDP (Week 12) call it **every step**.
- **Large clusters**: thousands of GPUs use it inside a server and between servers (next lecture).
- **Costs**: NVIDIA GPUs only; it shares the GPU with your compute; **every** GPU must make the call, or the others wait.

In our pipeline

T2 and Lab 4: benchmark NCCL collectives with torch.distributed. T3: DDP uses them underneath.

Why do we need NCCL?

Gradients live **on the GPUs**. NCCL sums them **GPU to GPU** over the fastest wire, instead of a slow trip through the CPU.

Source: PyTorch documentation, torch.distributed (v2.9). NVIDIA NCCL User Guide: Collective Operations; Communicators; CUDA Stream Semantics.

Learning goals for today

- **Explain** what NCCL and a **communicator** are, and what **all-reduce**, **reduce-scatter** and **all-gather** compute.
- **Build** the **ring all-reduce** step by step and derive its cost.
- **Explain** why that cost barely depends on the number of GPUs, and when a **tree** is better.
- **Test** a collective: correct values, real completion, same order on every rank.

A puzzle to start

Each of p GPUs holds 1 GB of gradients.
Goal: every GPU gets the elementwise sum.
Every GPU has the same link speed to its neighbours.

Going from p = 2 to p = 64 GPUs, roughly how much longer does the sum take?

Answer

Less than 2× longer — for large messages. By the end of Part 2 you will be able to prove it.

Recap: collectives and the α–β model

- **rank**: the number 0, 1, ..., p − 1 of each process in a parallel job.
- **collective**: an operation that **every** rank in a group calls; together they compute one result.

$$T(n) = \alpha + n \,/\, \beta$$

Week 10

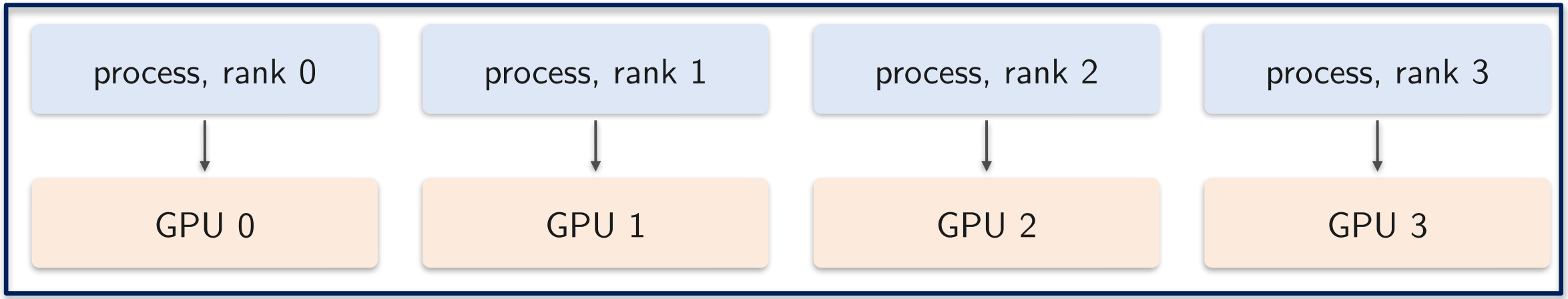
α: fixed **latency** per message (seconds). β: **bandwidth** of one link (bytes per second).

Part 1

What NCCL does

Who talks to whom, and what do they compute?

Communicators: ranks bound to GPUs



one communicator: 4 ranks, 4 GPUs

- **communicator**: the group of ranks that call collectives together. PyTorch: a **process group**.
- Rule: **one rank per GPU**; two ranks on one GPU may hang.

Source: NVIDIA NCCL User Guide, "Creating a Communicator".

NCCL from PyTorch

hello_nccl.py

```
import torch, torch.distributed as dist
dist.init_process_group("nccl")
r = dist.get_rank()
torch.cuda.set_device(r)    # one GPU per rank
x = torch.full((4,), float(r), device="cuda")
dist.all_reduce(x)          # sum, in place
print(r, x.tolist())
```

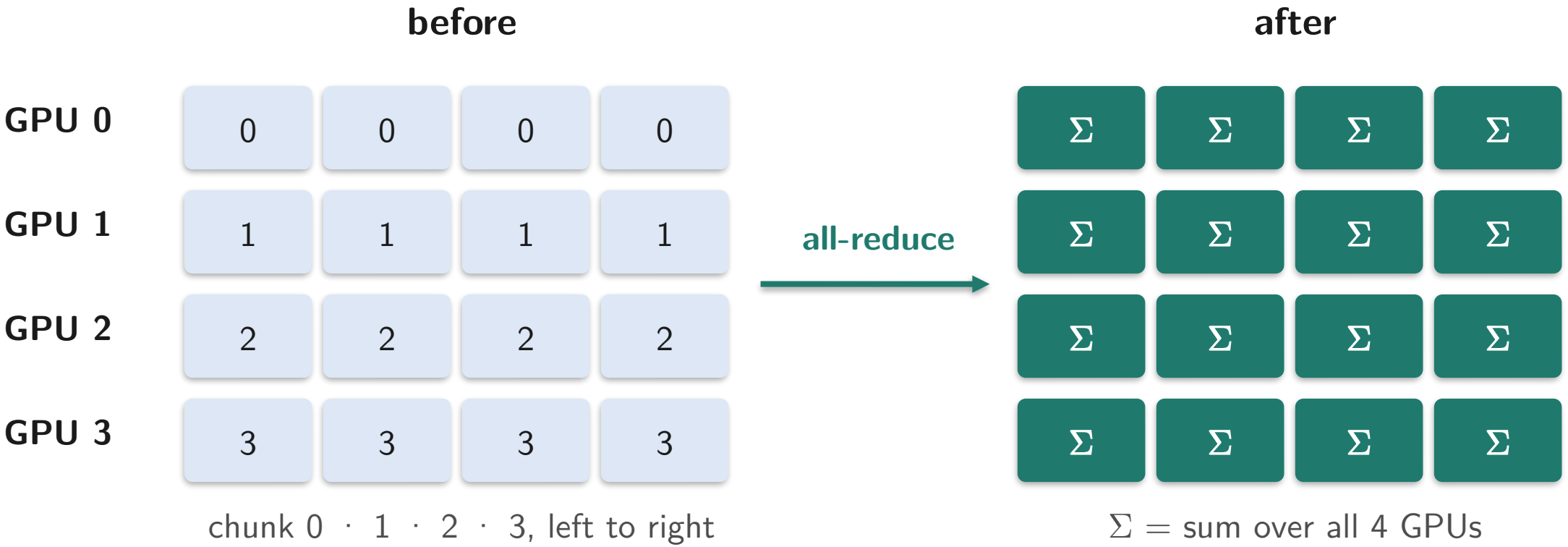
- One copy of the program **per GPU**.
- `all_reduce` sums over ranks, **in place**.
- 4 GPUs: every rank prints 6 $= 0 + 1 + 2 + 3$.

Key idea

Every rank runs the **same program**; only `r` differs.

Source: PyTorch documentation, torch.distributed (v2.9).

All-reduce: everyone gets the sum

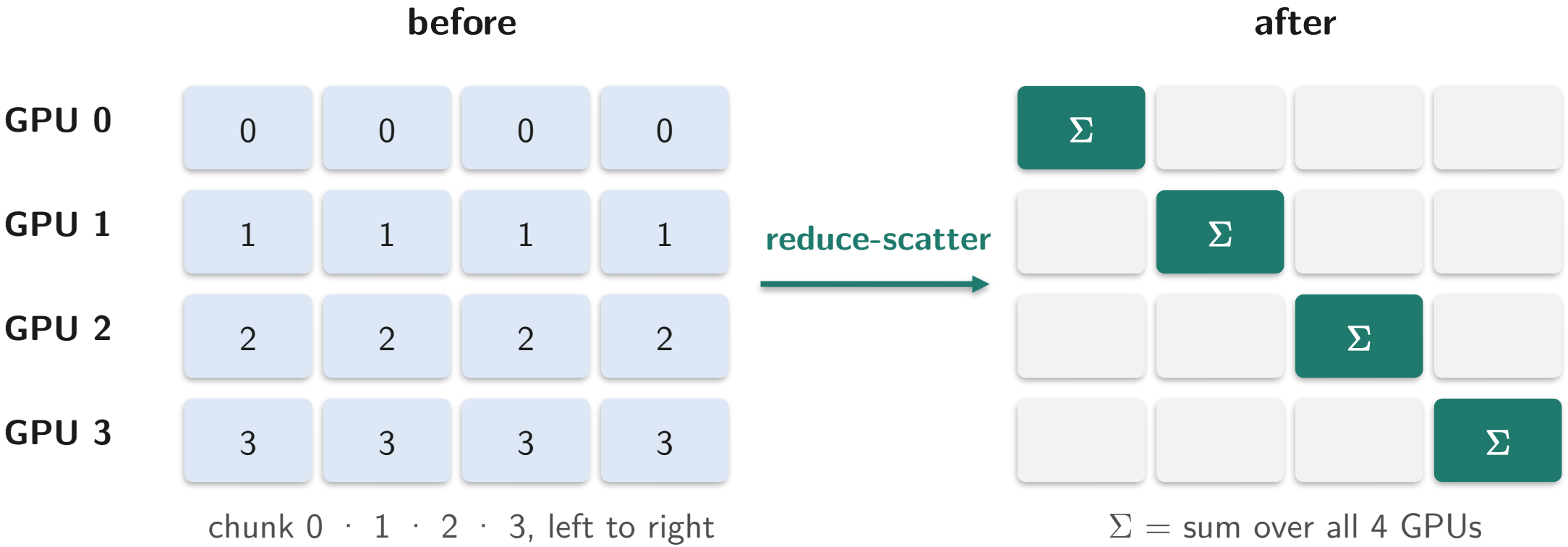


Definition

all-reduce: every GPU starts with its own vector and ends with the **sum of all of them**.

Source: NVIDIA NCCL User Guide: Collective Operations; Communicators; CUDA Stream Semantics.

Reduce-scatter: everyone gets one piece of the sum

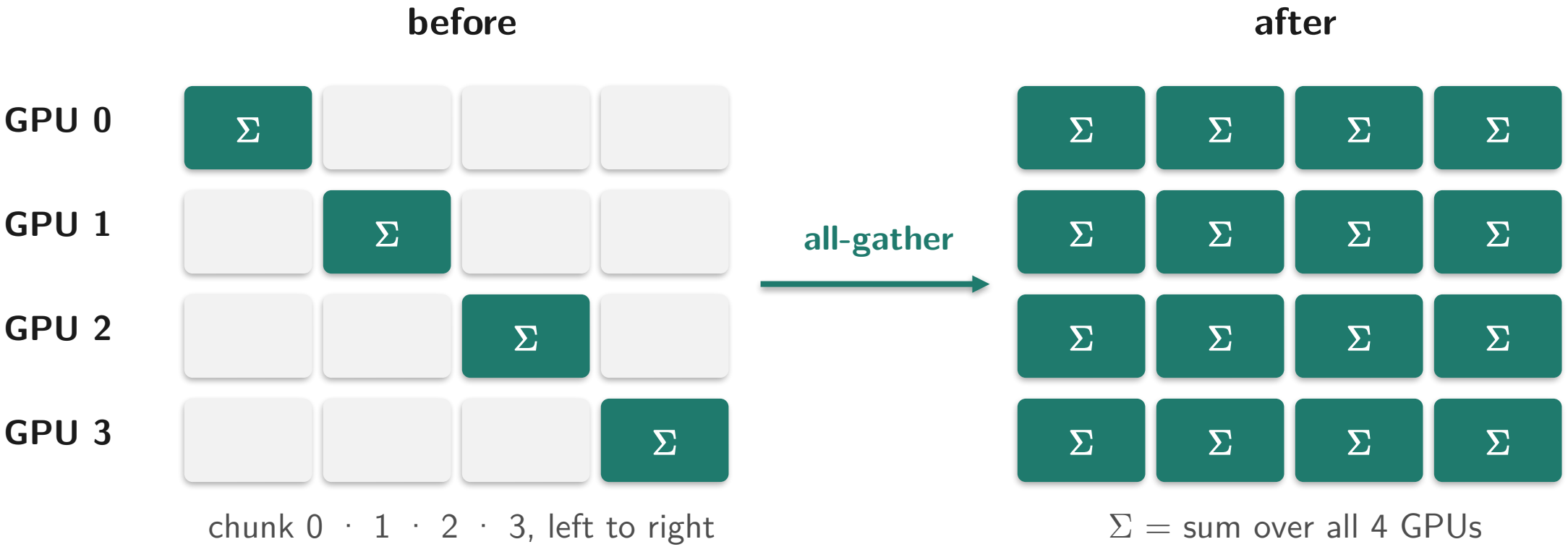


Definition

reduce-scatter: GPU k ends with **only chunk k of the sum**.

Source: NVIDIA NCCL User Guide: Collective Operations; Communicators; CUDA Stream Semantics.

All-gather: everyone collects all pieces



Definition

all-gather: GPU k starts with chunk k; every GPU ends with **all chunks**. Reduce-scatter, then all-gather = all-reduce.

Source: NVIDIA NCCL User Guide: Collective Operations; Communicators; CUDA Stream Semantics.

Check yourself

Two minutes with a neighbour.

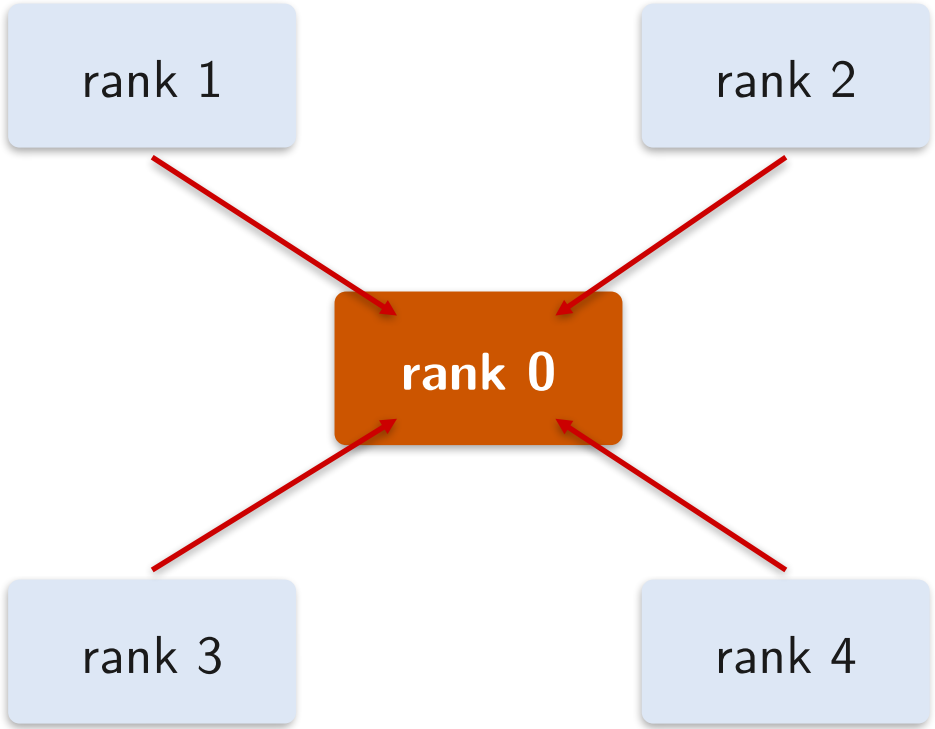
- What is a communicator? What does PyTorch call it?
- 4 GPUs reduce-scatter a vector of 4 chunks. Which part of the sum does GPU 2 end with?
- You forget `torch.cuda.set_device(r)`. What goes wrong with 4 GPUs?

Part 2

The ring all-reduce and its cost

How do p GPUs sum n bytes as fast as possible?

A first idea: gather to one GPU



$$\text{rank 0 receives } (p-1)\cdot n \text{ bytes}$$

$$\text{then sends } (p-1)\cdot n \text{ bytes}$$

$$T \approx 2(p-1)\cdot n \;/\; \beta$$

Watch out

8 GPUs, 1 GB each: T $\approx$ **31 ms**, and it grows **linearly** in p. One GPU's links do all the work.

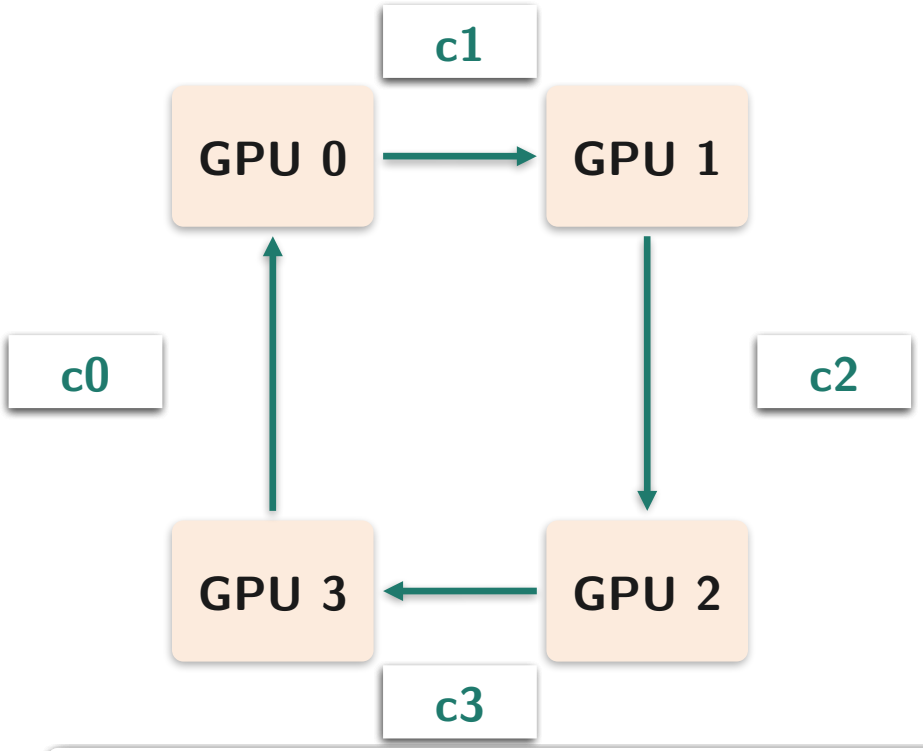
Source: H100 datasheet: NVLink 4, 900 GB/s bidirectional per GPU = 450 GB/s per direction.

The ring idea: split, then pass along

- Split each GPU's n bytes into **p chunks** of n/p bytes.
- Arrange the GPUs in a **ring**: GPU g sends to GPU g + 1.
- Every step, **every** GPU sends one chunk — all links busy at once.
- Phase 1 = **reduce-scatter**, phase 2 = **all-gather**: p − 1 steps each.

Source: Patarasuk & Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," JPDC 69(2), 2009.

Ring all-reduce, phase 1: reduce-scatter

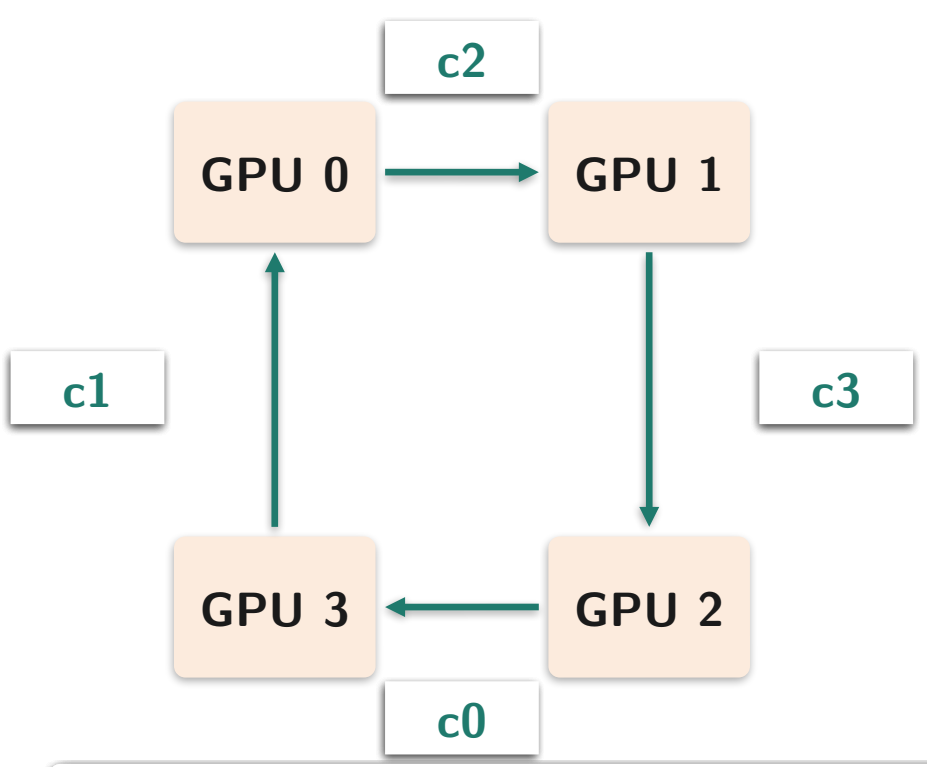


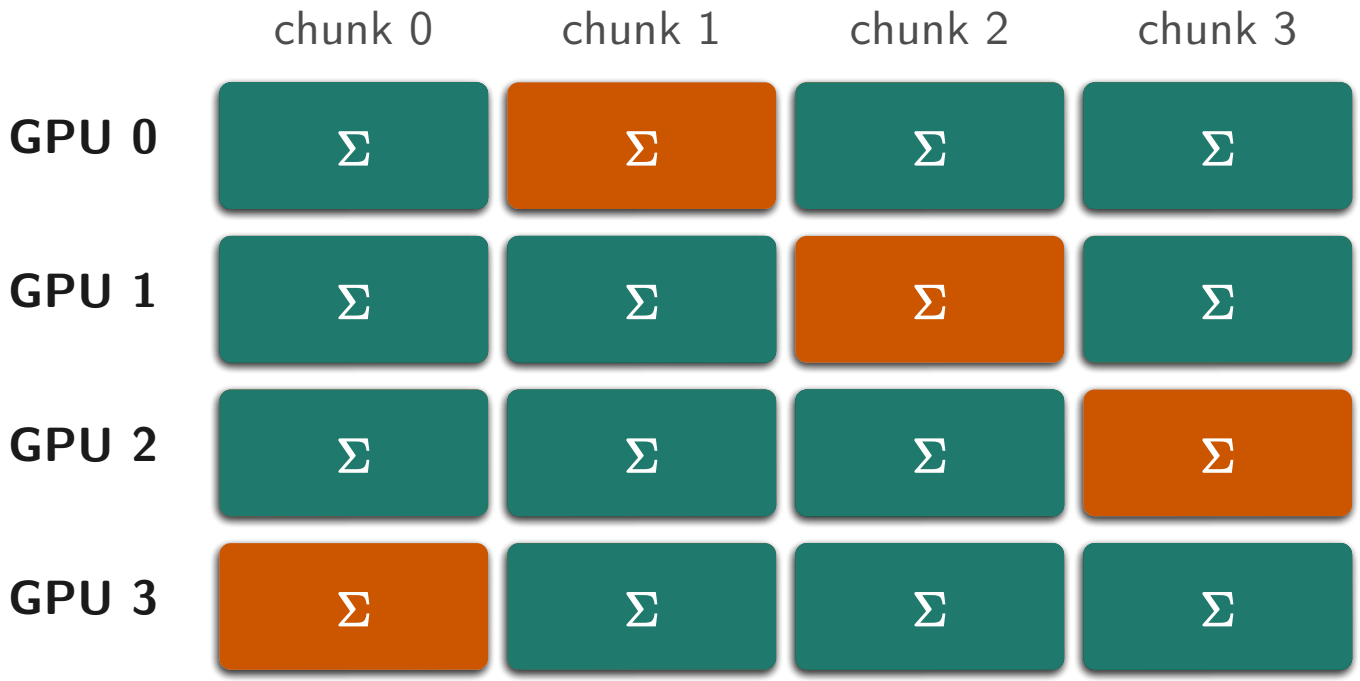
	chunk 0	chunk 1	chunk 2	chunk 3
GPU 0	Σ	2+3+0	3+0	0
GPU 1	1	Σ	3+0+1	0+1
GPU 2	1+2	2	Σ	0+1+2
GPU 3	1+2+3	2+3	3	Σ

Step 3: every GPU g sends chunk g − 3 to GPU g + 1, which adds it to its own copy

orange: just received · green: finished (Σ = all 4 GPUs)

Ring all-reduce, phase 2: all-gather





Step 3: every GPU g sends chunk g − 2 to GPU g + 1, which overwrites its copy

orange: just received · green: finished (Σ = all 4 GPUs)

Counting the cost of the ring

$$\text{steps:}\quad (p-1) + (p-1) = 2(p-1)$$

$$\text{one step:}\quad \alpha + (n \,/\, p) \,/\, \beta$$

$$T = 2(p-1)\ \alpha \ +\ 2(p-1)/p \cdot n \,/\, \beta$$

latency term

$2(p-1)\alpha$: counts steps

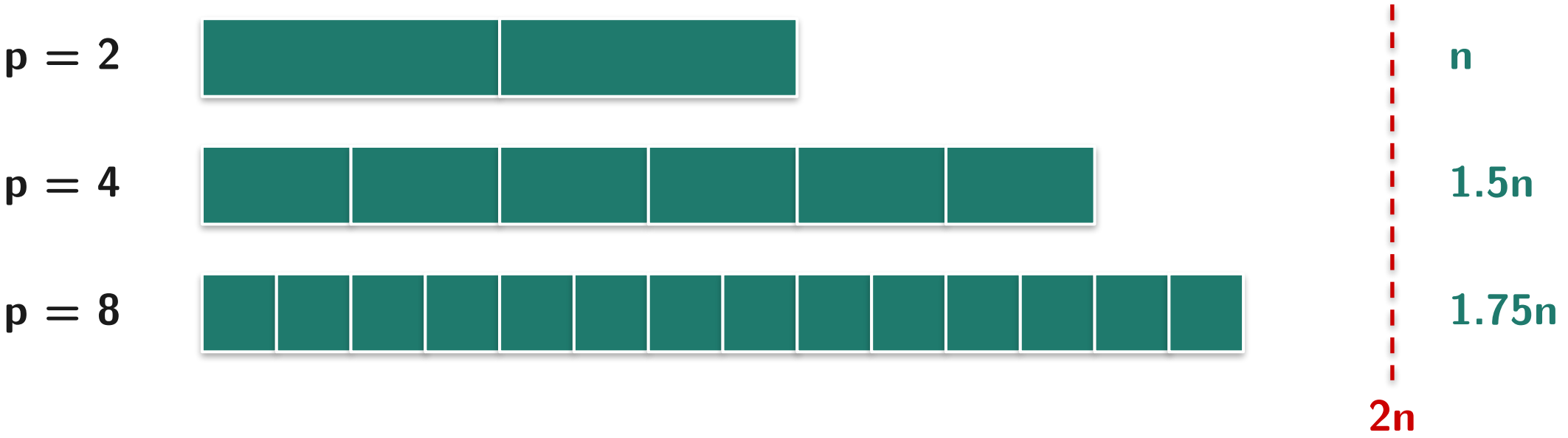
bandwidth term

$2(p-1)/p \cdot n/\beta$: counts bytes

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Why it barely depends on p

$$\text{bytes sent per GPU} = 2(p-1) \text{ chunks} \times n/\text{p} = 2(p-1)/p \cdot n$$



Key idea

More GPUs: **more steps**, but **smaller chunks**. Bytes per GPU stay **below 2n**.

Back to the puzzle

GPUs p	factor $2(p-1)/p$
2	1.00
8	1.75
64	1.97

Answer

Less than 2× from 2 to 64 GPUs (large messages). The latency term $2(p-1)\alpha$ **does** grow with p.

Source: Factor computed from the ring cost; Patarasuk & Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," JPDC 69(2), 2009.

No algorithm beats the ring's bandwidth term

RESULT

Every all-reduce here needs $\mathbf{T \geq 2(p-1)/p \cdot n/\beta}$, and the ring achieves it.

- Each entry: $\mathbf{p-1}$ sends to add it up, $\mathbf{p-1}$ more to deliver the sum.
- So $\geq 2(p-1) \cdot n$ bytes in all: the busiest GPU sends $\geq \mathbf{2(p-1)/p \cdot n}$.

Key idea

Better algorithms can only cut the **latency** term — fewer steps.

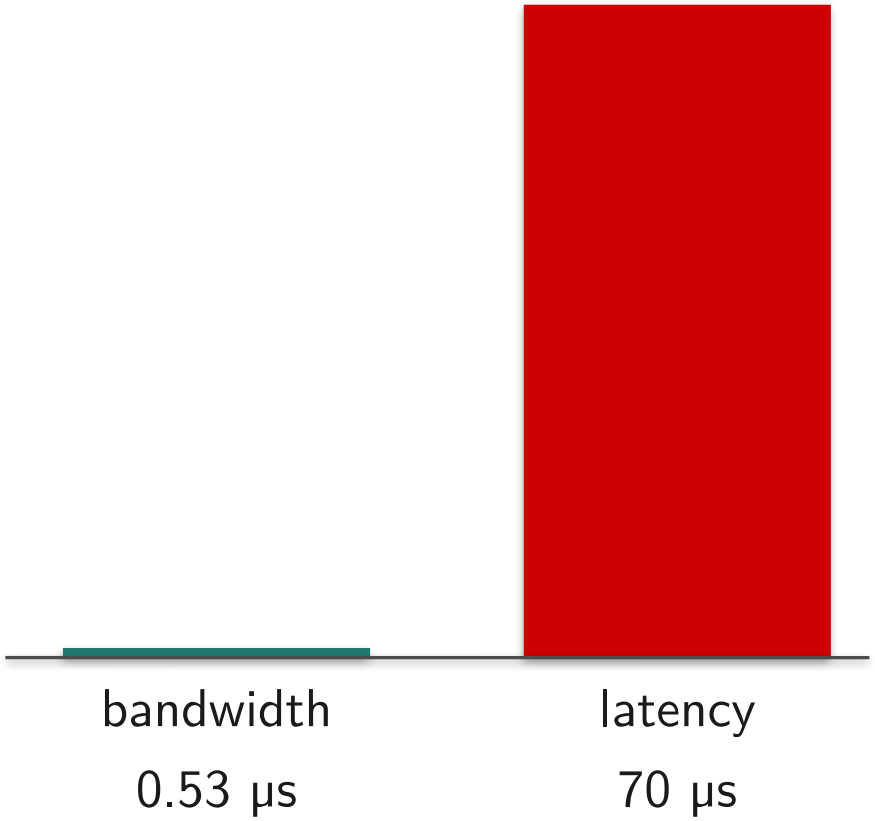
Source: Patarasuk & Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," JPDC 69(2), 2009.

Small messages: latency wins

- Our MLP's gradient is small: **137 KB**.
- On 8 GPUs: the bytes take almost no time; the 14 steps do.

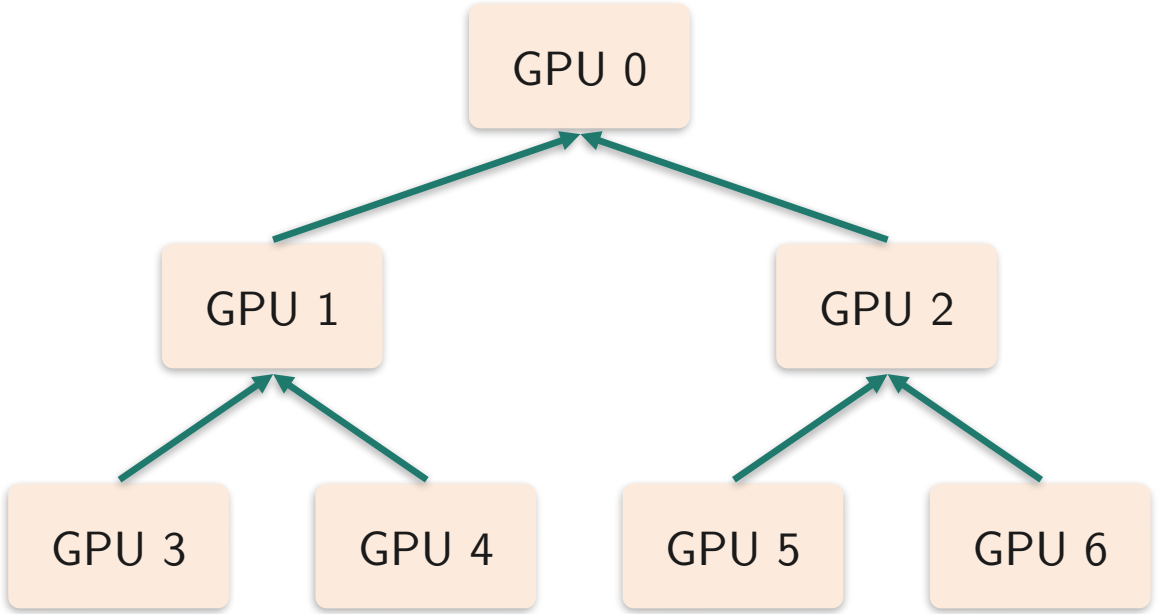
Watch out

Small messages are **latency-bound**: count steps, not bytes.



Source: $\beta = 450$ GB/s (H100 datasheet); $\alpha = 5$ µs per step (assumed).

Trees: fewer steps for small messages



step 1: partial sums move up

step 2: partial sums move up

$$\text{tree: } 2 \log_2 p \text{ steps}$$

$$\text{ring: } 2(p-1) \text{ steps}$$

$$p = 64\text{: } 12 \text{ versus } 126$$

Key idea

Trees: fewer **steps** (small messages).
Rings: fewer **bytes** (large messages).
NCCL picks per call.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Check yourself

Two minutes with a neighbour.

- 4 GPUs, n $=$ 400 MB, $\beta = 450$ GB/s. Ignoring latency, how long does a ring all-reduce take?
- From 8 to 64 GPUs: what happens to each term of the ring cost?
- 64 GPUs all-reduce a 4 KB tensor. Ring or tree? Why?

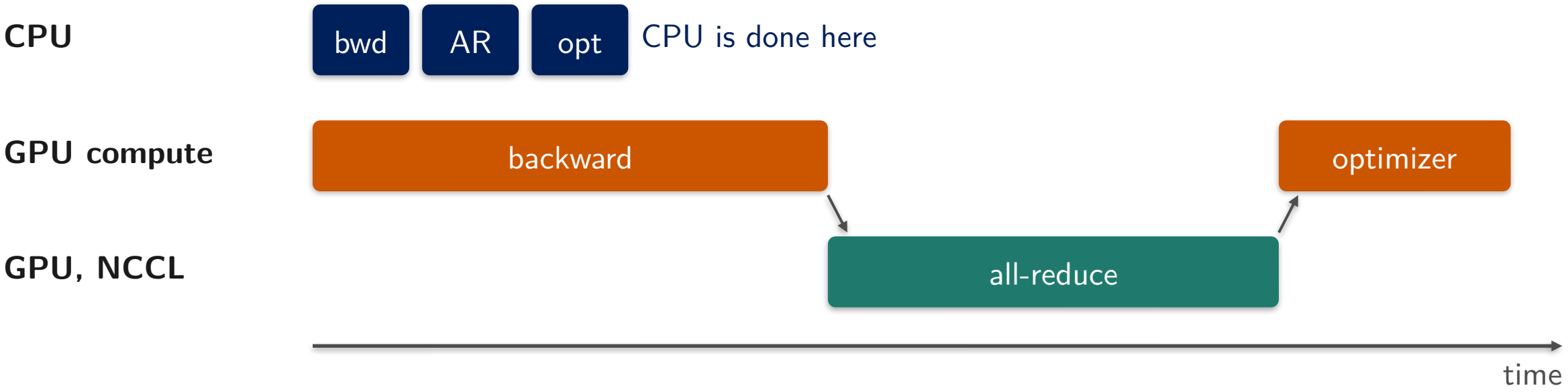
Part 3

Ordering and correct testing

When is a collective finished — and why do programs hang?

A call returns before the sum is done

A **CUDA stream** is a queue of GPU work: the CPU adds work and moves on.



Watch out

When `all_reduce` returns, the sum is **queued**, not **done**. Later GPU work still sees the right order.

Source: NCCL User Guide, "CUDA Stream Semantics"; PyTorch documentation, torch.distributed (v2.9).

Timing a collective correctly

```
for _ in range(5):
    dist.all_reduce(x)            # warm up
torch.cuda.synchronize()
t0 = time.perf_counter()
for _ in range(20):
    dist.all_reduce(x)
torch.cuda.synchronize()          # wait for GPU
t = (time.perf_counter() - t0) / 20
```

- **Warm up**: the first call sets up NCCL.
- **Synchronize** before reading the clock.
- **Many runs**, then divide.

Watch out

Without `synchronize()` you time the **queueing**, not the transfer.

Source: PyTorch documentation, torch.distributed (v2.9).

Ordering: every rank, same calls, same order

rank 0

```
all_reduce(a)
all_reduce(b)
```

rank 1

```
all_reduce(b)   # swapped!
all_reduce(a)
```

mismatched: rank 0's a is paired with rank 1's b

- Same size: a **silently wrong** sum (a + b).
- Different sizes: usually a **hang** until the timeout.

Watch out

Collectives are matched **by position**. Every rank must issue the same sequence.

Source: PyTorch documentation, torch.distributed (v2.9).

How ordering bugs sneak in

hangs

```
loss = compute_loss(batch)
if rank == 0:                          # BUG
    dist.all_reduce(loss)              # only rank 0
    print(loss.item() / p)
```

Ranks 1, 2, 3 never call `all_reduce`: rank 0 **waits forever**.

FIX

Make collectives **unconditional**: every rank calls `all_reduce`; only the `print` stays inside `if rank == 0`.

Watch out

Also watch for ranks with **different numbers of batches**: one rank makes an extra call.

Source: PyTorch documentation, torch.distributed (v2.9).

Bus bandwidth: compare with the link

You all-reduce n bytes on p GPUs and measure the time t.

$$\text{busbw} = (n \ / \ t) \cdot 2(p-1)/p$$

Why the factor: each GPU really moves $2(p-1)/p \cdot n$ bytes, not n.

In practice

Compare **busbw** with the link: e.g. **438 GB/s** is 97% of NVLink's **450 GB/s**.

Source: NVIDIA nccl-tests, doc/PERFORMANCE.md (algorithm and bus bandwidth). NVLink: H100 datasheet.

Check yourself

Two minutes with a neighbour.

- You time 20 all-reduces **without** `synchronize()`. What did you measure?
- Rank 3 has 101 batches, the others 100. What happens at the end?
- 8 GPUs all-reduce 0.5 GB in 2.5 ms. What is the busbw?

IN-CLASS BUILD

Test NCCL results and completion

Work in pairs. This is the T2 checkpoint test.

Exercise: a correctness test

all-reduce test

```
p, r = dist.get_world_size(), dist.get_rank()
i = torch.arange(1024, device="cuda",
                 dtype=torch.float32)
x = r + i                              # rank-dependent
dist.all_reduce(x)
want = p * (p - 1) / 2 + p * i  # closed form
assert torch.equal(x, want), r
```

Use **integer** values in FP32, so `torch.equal` is exact. Test p = 2 and p = 4.

QUESTIONS

(1) Why is x[i] = r + i a good input? What is the sum for p = 4?

(2) After a reduce-scatter of the same x, what should rank r hold?

(3) The test passes, and you time it: 2 µs for 1 GB. What is missing?

Exercise: solutions

- (1) It differs per rank **and** per position, so any mix-up shows. p = 4: **6 + 4i**.
- (2) **Chunk r** of that sum: the same formula on chunk r's positions.
- (3) `torch.cuda.synchronize()` before stopping the clock.

Check

A test that only checks "no crash" would pass a **swapped-order** bug. Check **values**.

How to test it

- Inputs with a **closed-form** answer; check with `torch.equal`.
- Run on $\mathbf{p = 2}$ and $\mathbf{p = 4}$.
- Test **all three**: all-reduce, reduce-scatter, all-gather.
- **Synchronize** before reading the clock.
- Sweep sizes; compare **busbw** with the link.

In our pipeline

T2 checkpoint: correct results for all three collectives at $p = 2$ and 4, plus a timed sweep.

Source: NVIDIA nccl-tests, doc/PERFORMANCE.md (algorithm and bus bandwidth).

Key takeaways

- NCCL runs collectives as **GPU programs**; a **communicator** has one rank per GPU.
- **All-reduce = reduce-scatter + all-gather**.
- Ring: $T = \mathbf{2(p-1)}\alpha + \mathbf{2(p-1)/p} \cdot \mathbf{n}/\beta$: more steps, smaller chunks, under 2n bytes per GPU.
- Small messages are **latency-bound**; trees need fewer steps.
- Calls are **queued**: synchronize before timing; same calls, **same order** on every rank.

Next time, and readings

When	What
This week	T2 checkpoint: correct collectives on small tensors
Lecture 2 (this week)	Multi-server communication: topology, launch, timeouts, α–β fit
Week 12	Data parallelism and sharding (DDP, ZeRO/FSDP)

READ BEFORE LECTURE 2

NCCL User Guide: *Collective Operations*, *Communicators*, *CUDA Stream Semantics*.
PyTorch *Distributed Overview*. Thakur et al., IJHPCA 2005 (Sections 1–4).